# Summer Training Program **WEEK 2 - DAY 9**

Topics discussed

- More on constructors
- Functions in classes
- Circular doubly linked list- hard code and dynamic coding

More on functions in class:

1. Inside the constructor we can give default values to all variables.

```
class User:

    def __init__(self, name=None, phone=None, email=None):
        self.name = name
        self.phone = phone
        self.email = email
        print('[User] [init] User Object Constructed', self)
```

2. We can also input values using constructor:

```
    def __init__(self, name=input('enter name: '),
                       phone=input('enter phone: '),
                       email= input('enter email: ')):
        pass
```

Defining a function in class other than constructor:

1. The first variable of all functions has to be 'self' and all the variables of the class are accessed by using self with a dot(membership) operator.

```
    def input_user_details(self):
        self.name = input('Enter Name: ')
        self.phone = input('Enter Phone: ')
        self.email = input('Enter Email: ')
```

One more example:

```
    def show_user_details(self):
        print('~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~')
        print(self.name, self.phone, self.email)
        print('~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~')
```

2. Now when we print the object associated with the class we get:

```
user1 = User()
print(vars(user1))
```

```
<__main__.User object at 0x000001FF0AB06BA0>
```

3. But when we print a class, we get:

```
33    print(vars(User))
```

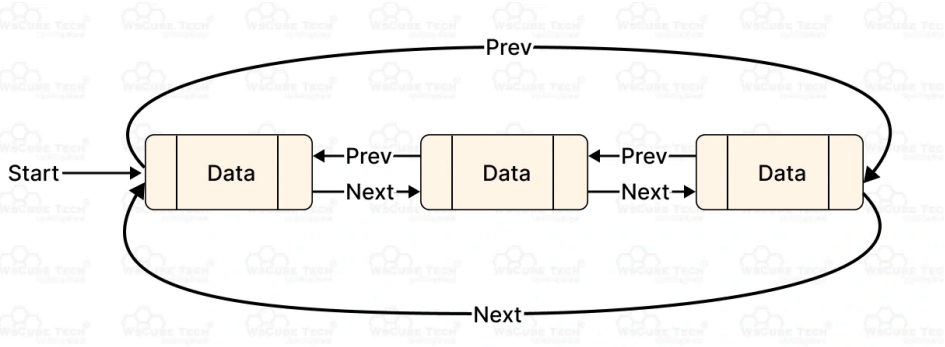
```
{'__module__': '__main__', '__firstlineno__': 2, '__doc__': "def __
init__(self, name=None, phone=None, email=None):\nself.name = name\
nself.phone = phone\nself.email = email\nprint('[User] [init] User
Object Constructed', self)", 'input_user_details': <function User.i
nput_user_details at 0x0000028D3127F060>, 'show_user_details': <fun
ction User.show_user_details at 0x0000028D312980E0>, '__static_attr
ibutes__': ('email', 'name', 'phone'), '__dict__': <attribute '__di
ct__' of 'User' objects>, '__weakref__': <attribute '__weakref__' o
f 'User' objects>}
```

- We get the information of the constructor and all the functions and all the other hidden attributes.
- This tells us that when objects are created in memory, functions are not created with it. [functions are the property of the class not the object]
- Functions are stored in the permanent memory.
- We have another section, other than the stack and heap called the code section, where function definitions are stored.

Circular doubly linked list:

It is a special data structure with the following features,

- Each element is connected to its previous and its next element.
- The last element is also connected to the first one.
- By connected we mean that they stores the address of one another.




Eg:

A spotify playlist

The recommendations window (slide show) in OTT platforms

Songs on a CD.

>>Now we take the example of a playlist and hardcode a doubly linked list:

1. We start by identifying the object and its attributes:
   - So the object is a song, attributes are name, artist and duration.
   - We also need two more variables to store the previous and next song.

2. Coding the constructor:

```
class Song:
    def __init__(self,title,artist,duration, next_song, previous_song ):
        self.title= title
        self.artist= artist
        self.duration= duration
        self.next_song= None
        self.previous_song = None
```

3. Now we will generate objects (songs)
   For eg: we code these 5 objects:

```
#make 5 objects i.e. songs

song1= Song(title= 'is it a crime',
            artist= 'Sade',
            duration= 4.0,
            previous_song=None,
            next_song=None)

song2= Song(title= '7 seconds',
            artist= 'neneh cherry',
            duration= 5.0,
            previous_song=None,
            next_song=None)

song3= Song(title= 'no ordinary love',
            artist= 'Sade',
            duration= 4.5,
            previous_song=None,
            next_song=None)

song4= Song(title= 'paradise',
            artist= 'Sade',
            duration= 6.0,
            previous_song=None,
            next_song=None)

song5= Song(title= 'iwicked game',
            artist= 'chris isaak',
            duration= 5.0,
            previous_song=None,
            next_song=None)
```

   Now we have created these 5 objects which are the actual songs in the playlist.

   Now we can create a show_song() function that can help us print the details of the songs.

   Implicit calling:

   song1.show_song()
   This is implicit calling because the cpu automatically understands that we want to execute the show_song() function for song1 which is an object of the class Song

   Explicit calling

   Song.show_song(song1)
   Here we explicitly mention the class of the object.

4. Coding the show function:

```
    def show_song(self):
        print('~~~~~~~')
        print('title: ', self.title)
        print('artists: ', self.artist)
        print('duration: ', self.duration)
        print('next song: ', self.next_song)
        print('previous song: ', self.previous_song)
        print('~~~~~~~')
```

5. Manually establishing the links between objects:
   We can do this simply by assigning the next and previous values:

```
song1.next_song=song2
song2.next_song=song3
song3.next_song=song4
song4.next_song=song5
song5.next_song=song1

song1.previous_song=song5
song2.previous_song=song1
song3.previous_song=song2
song4.previous_song=song3
song5.previous_song=song4
```

6. At the last we can print this circular doubly linked list:

CODE:

```
song1.show_song()
song2.show_song()
song3.show_song()
song4.show_song()
song5.show_song()
```

OUTPUT:

```
~~~~~~~
title:  is it a crime
artists:  Sade
duration:  4.0
next song:  <__main__.Song object at 0x00000170D7748A50>
previous song:  <__main__.Song object at 0x00000170D752A8B0>
~~~~~~~
~~~~~~~
title:  7 seconds
artists:  neneh cherry
duration:  5.0
next song:  <__main__.Song object at 0x00000170D7748B90>
previous song:  <__main__.Song object at 0x00000170D74B6A50>
~~~~~~~
~~~~~~~
title:  no ordinary love
artists:  Sade
duration:  4.5
next song:  <__main__.Song object at 0x00000170D752A780>
previous song:  <__main__.Song object at 0x00000170D7748A50>
~~~~~~~
~~~~~~~
title:  paradise
artists:  Sade
duration:  6.0
next song:  <__main__.Song object at 0x00000170D752A8B0>
previous song:  <__main__.Song object at 0x00000170D7748B90>
~~~~~~~
~~~~~~~
title:  iwicked game
artists:  chris isaak
duration:  5.0
next song:  <__main__.Song object at 0x00000170D74B6A50>
previous song:  <__main__.Song object at 0x00000170D752A780>
~~~~~~~
```

Now we code the circular doubly linked list dynamically:

1. We change the logic of printing the songs manually
   For this we use a loop:

```
print('FORWARD TRAVERSING:')
song = song1

while True:
    song.show_song()
    song = song.next_song

    if song == song1:
        break


print('BACKWARD TRAVERSING:')
song = song5

while True:
    song.show_song()
    song = song.previous_song

    if song == song5:
        break
```

   This logic can work for both backward and forward traversing of the songs.
   We use a simple while loop which prints the songs from the first song until we reach the first song ( in case of forward traversing) and the last song (in case of backward traversing).

2. Dynamically linking the songs:
   For this we create a new object called Songlist, this is the class that actually implements the circular doubly linked list, it has 3 variables:
   head, tail and size where
   Head: the first song
   Tail: the last song
   Size: the total number of songs

```
# class SongList:
class CircularDoublyLinkedList:

    def __init__(self):
        self.head = None
        self.tail = None
        self.size = 0
        print('[CircularDoublyLinkedList] [init] Object Constrcuted', self)
```

   Initially the variables head and tail are set to None and the value of size is set to 0.
   This is the situation when the playlist is empty, i.e. no songs exist, i.e. no objects have been created.

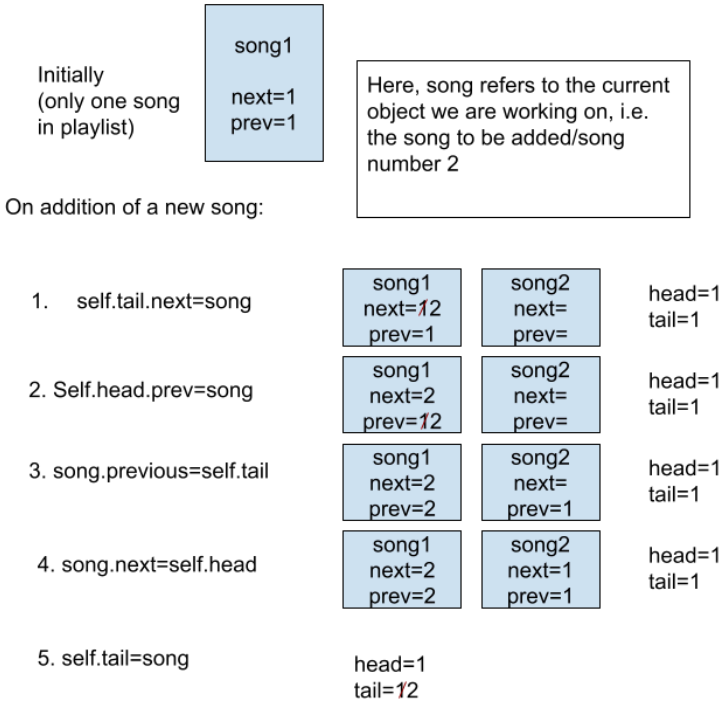
3. Now we implement the logic to add objects(songs) to this empty object:
   For this we make a function add inside the class CircularDoublyLinkedList:

```
11      def add(self, element):
12
13          self.size += 1
14
15          if self.head == None:
16              self.head = element
17              self.tail = element
18          else:
19              self.tail.next_song = element
20              self.head.previous_song = element
21              element.previous_song = self.tail
22              element.next_song = self.head
23              self.tail = element
```

   According to this logic,

   - The size attribute is incremented by one whenever an element is added.
   - First the condition is checked if the playlist is empty (line 15)
     if yes, then the head is assigned to the added element as well as the tail.
   - If the list is not empty, in this case
     The next_song variable of last song(tail) is assigned to new song
     And the previous_song variable of the first song(head) is assigned to the newsong too.
     The previous song of the new song is assigned to tail.
     And the next song of the new song(now the last) is assigned to the head.
     And then at last the tail is updated to store the newly added song.

   Diagramatically:



4. Now assigning a reference variable to the object each time takes up a new memory block in the RAM, to make this more efficient, we can simply input the song details as it is:

```
song_list= CircularDoublyLinkedList()

song_list.add(element= Song(title= '1. is it a crime',
                artist= 'Sade',
                duration= 4.0,
                previous_song=None,
                next_song=None))
```

Now if we finally print the list of songs we get:

```
Title: 1. is it a crime
Artist: Sade
Duration: 4.0
HashCode: <class_Song.Song object at 0x000001FD7B4E6CF0>
next_song: <class_Song.Song object at 0x000001FD7B7E8A50>
previous_song: <class_Song.Song object at 0x000001FD7B55B230>
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
Title: 2. 7 seconds
Artist: neneh cherry
Duration: 5.0
HashCode: <class_Song.Song object at 0x000001FD7B7E8A50>
next_song: <class_Song.Song object at 0x000001FD7B7E8B90>
previous_song: <class_Song.Song object at 0x000001FD7B4E6CF0>
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
Title: 3. no ordinary love
Artist: Sade
Duration: 4.5
HashCode: <class_Song.Song object at 0x000001FD7B7E8B90>
next_song: <class_Song.Song object at 0x000001FD7B55A9E0>
previous_song: <class_Song.Song object at 0x000001FD7B7E8A50>
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
Title: 4. paradise
Artist: Sade
Duration: 6.0
HashCode: <class_Song.Song object at 0x000001FD7B55A9E0>
next_song: <class_Song.Song object at 0x000001FD7B55B230>
previous_song: <class_Song.Song object at 0x000001FD7B7E8B90>
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
Title: 5. wicked game
Artist: chris isaak
Duration: 5.0
HashCode: <class_Song.Song object at 0x000001FD7B55B230>
next_song: <class_Song.Song object at 0x000001FD7B4E6CF0>
previous_song: <class_Song.Song object at 0x000001FD7B55A9E0>
```

Hence, we have dynamically coded a circular doubly linked list.

**ASSIGNMENT:**

**Code the following functions for a doubly linked list:**

- **Add in between**
- **Add in front**
- **Delete last**
- **Delete front**
- **Delete specific element**

**GURASSIS KAUR**

**g.assiskaur@gmail.com**